

TD : Tests Unitaires et TDD en PHP (PHPUnit & Pest)

Table des matières

1) Problématique	1
2) Prérequis & Installation	1
3) Exercices : Implémentation en démarche TDD	3
4) Refactoring du code et non-régression	6
5) Conclusion	8

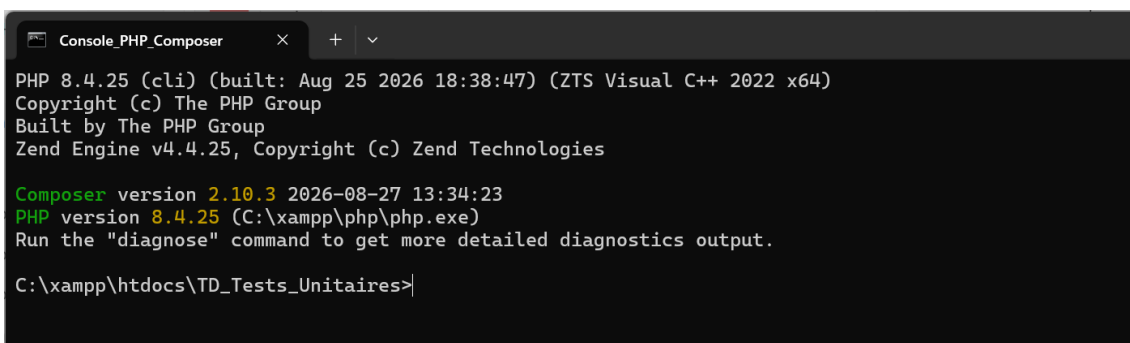
1) Problématique

Dans le développement d'applications professionnelles, tester manuellement son code via un navigateur ou un formulaire devient rapidement insuffisant dès que le projet prend de l'ampleur. L'ajout de nouvelles fonctionnalités risque à tout moment de créer des régressions ou des effets de bord sur le code existant. L'approche TDD (Test Driven Development) répond à cette problématique en inversant la logique traditionnelle : on commence par écrire les jeux de tests avant même de développer la moindre ligne de code métier. L'objectif de ce TD est de découvrir les tests unitaires automatisés en PHP. Après avoir étudié les principes fondamentaux sans framework lors du cours précédent, nous allons installer et mettre en œuvre les deux outils de référence de l'écosystème PHP : le framework classique PHPUnit et le framework moderne Pest. Nous reproduirons l'exemple pratique de fabrication de potions magiques en validant chaque étape d'implémentation et de refactoring grâce à des tests automatisés.

2) Prérequis & Installation

1) Vérification de l'environnement PHP et Composer

Afin de mener à bien ce TP, nous avons besoin de deux outils essentiels sur notre machine : PHP (version 8.2 ou supérieure) et le gestionnaire de dépendances Composer. Dans le terminal, on commence d'abord par vérifier que PHP et Composer sont bien reconnus et accessibles dans notre variable d'environnement PATH :



```
Console_PHP_Composer  X  +  v
PHP 8.4.25 (cli) (built: Aug 25 2026 18:38:47) (ZTS Visual C++ 2022 x64)
Copyright (c) The PHP Group
Built by The PHP Group
Zend Engine v4.4.25, Copyright (c) Zend Technologies

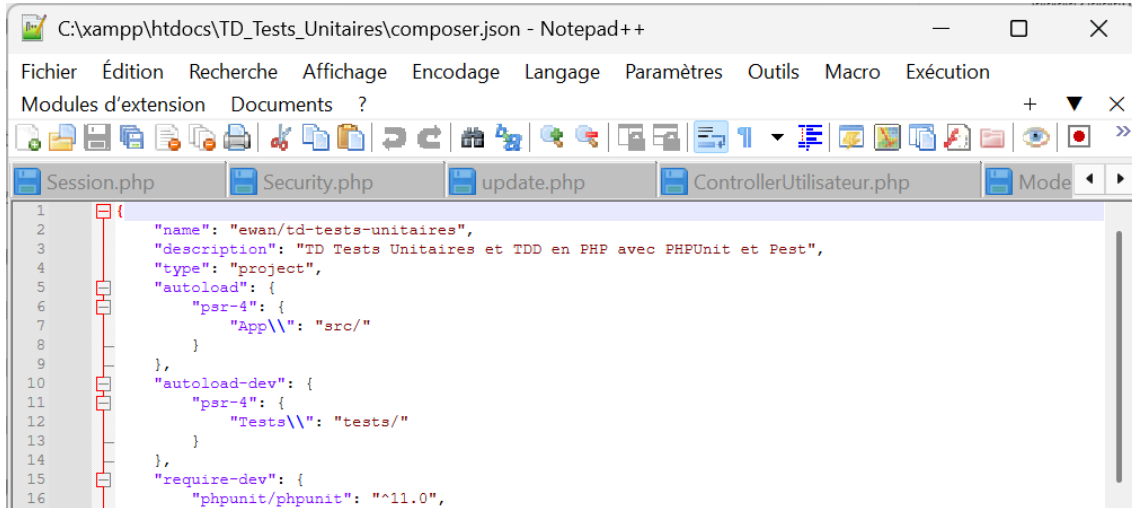
Composer version 2.10.3 2026-08-27 13:34:23
PHP version 8.4.25 (C:\xampp\php\php.exe)
Run the "diagnose" command to get more detailed diagnostics output.

C:\xampp\htdocs\TD_Testes_Unitaires>
```

On constate que PHP 8.4.25 et Composer 2.10.3 sont parfaitement opérationnels.

2) Configuration du projet via composer.json

On crée un dossier dédié nommé TD_Tests_Unitaires dans notre environnement de travail. On initialise ensuite la configuration du projet avec un fichier composer.json définissant l'autoloading PSR-4 (qui permet de charger automatiquement nos classes du dossier src/ sous le namespace App) ainsi que les dépendances de développement require-dev :



```
1 {
2     "name": "ewan/td-tests-unitaires",
3     "description": "TD Tests Unitaires et TDD en PHP avec PHPUnit et Pest",
4     "type": "project",
5     "autoload": {
6         "psr-4": {
7             "App\\": "src/"
8         }
9     },
10    "autoload-dev": {
11        "psr-4": {
12            "Tests\\": "tests/"
13        }
14    },
15    "require-dev": {
16        "phpunit/phpunit": "^11.0",
```

3) Installation et vérification de PHPUnit

On installe PHPUnit en dépendance de développement grâce à la commande `composer require --dev phpunit/phpunit`. Composer télécharge alors PHPUnit ainsi que l'ensemble de ses composants dans le dossier `vendor/`. On vérifie ensuite que l'exécutable est disponible dans `vendor/bin/phpunit` :



```
PHPUnit 11.5.56 by Sebastian Bergmann and contributors.

C:\xampp\htdocs\TD_Tests_Unitaires>
```

PHPUnit 11.5.56 est correctement installé et prêt à l'emploi.

4) Installation et initialisation du framework Pest

Pest est un framework de test PHP moderne bâti au-dessus de PHPUnit. Il apporte une syntaxe plus fluide, plus lisible et grandement inspirée des frameworks de test JavaScript comme Jest. On installe Pest via Composer avec la commande `composer require pestphp/pest --dev --with-all-dependencies`, puis on lance son initialisation avec `./vendor/bin/pest --init` :



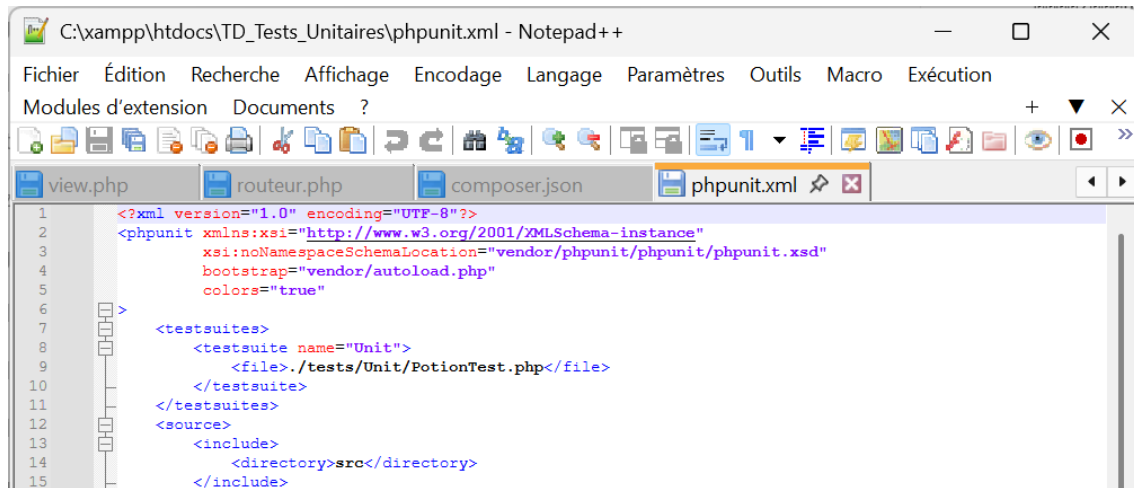
```
Pest Testing Framework 3.8.7.

C:\xampp\htdocs\TD_Tests_Unitaires>
```

Pest 3.8.7 est désormais configuré avec succès.

5) Fichier de configuration phpunit.xml

L'initialisation génère le fichier de configuration phpunit.xml à la racine de notre projet. Ce fichier configure l'environnement d'exécution des tests, charge l'autoloader Composer vendor/autoload.php, active l'affichage en couleur dans le terminal et définit la suite de tests à exécuter :



```
<?xml version="1.0" encoding="UTF-8"?>
<phpunit xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:noNamespaceSchemaLocation="vendor/phpunit/phpunit/phpunit.xsd"
  bootstrap="vendor/autoload.php"
  colors="true"
>
  <testsuites>
    <testsuite name="Unit">
      <file>./tests/Unit/PotionTest.php</file>
    </testsuite>
  </testsuites>
  <source>
    <include>
      <directory>src</directory>
    </include>
  </source>
</phpunit>
```

3) Exercices : Implémentation en démarche TDD

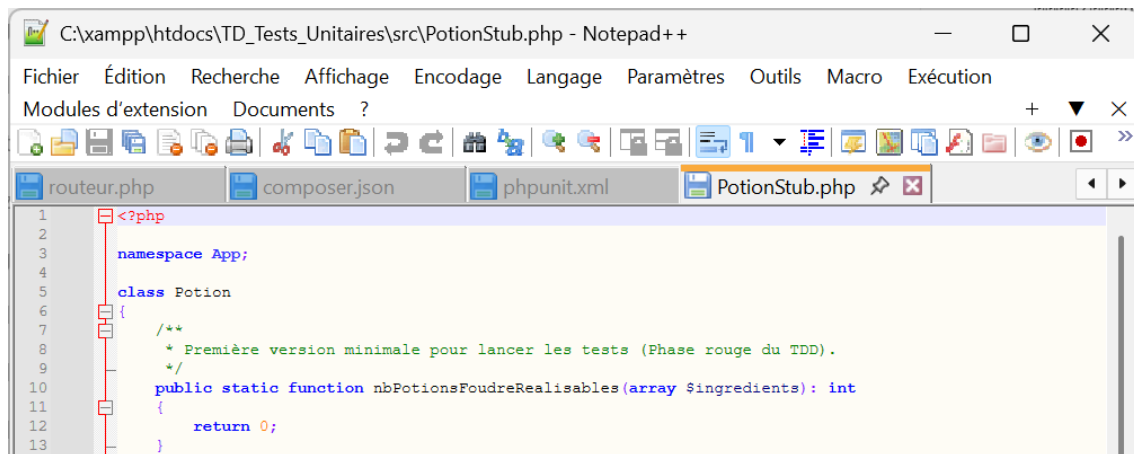
1) Présentation du problème et définition des cas de test

Comme dans l'exemple du cours, nous souhaitons implémenter la fonction nbPotionsFoudreRealisables(array \$ingredients): int qui calcule le nombre de potions de foudre fabriquables selon les stocks disponibles. La recette nécessite précisément 2 nuages ténébreux ('nuages_tenebreux' => 2) et 1 poussière de fée ('poussiere_fee' => 1). En démarche TDD, nous définissons d'abord l'intégralité des 9 cas d'usage possibles :

- Test 1 : Ingrédients suffisants pour 1 potion (2 nuages, 1 poussière -> attendu : 1)
- Test 2 : Premier ingrédient insuffisant (1 nuage, 1 poussière -> attendu : 0)
- Test 3 : Quantités multiples pour 4 potions (8 nuages, 4 poussières -> attendu : 4)
- Test 4 : Manque le premier ingrédient (1 poussière -> attendu : 0)
- Test 5 : Manque le deuxième ingrédient (2 nuages -> attendu : 0)
- Test 6 : Ingrédients tiers inutiles (queue de lézard, eau de jouvence -> attendu : 0)
- Test 7 : Aucun ingrédient fourni ([]) -> attendu : 0)
- Test 8 : Premier ingrédient en excédent mais deuxième limitant (250 nuages, 1 poussière -> attendu : 1)
- Test 9 : Deuxième ingrédient en excédent mais premier limitant (2 nuages, 120 poussières -> attendu : 1)

2) Phase rouge du TDD : Écriture des tests et fonction minimale stub

La première phase du cycle TDD (Red) consiste à coder une version minimale de la fonction, appelée stub, qui renvoie simplement 0 :



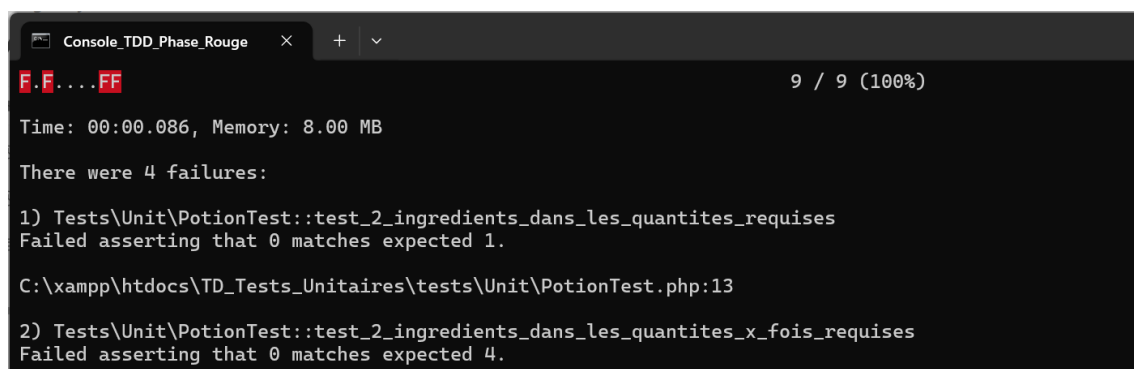
```
1 <?php
2
3 namespace App;
4
5 class Potions
6 {
7     /**
8      * Première version minimale pour lancer les tests (Phase rouge du TDD).
9      */
10    public static function nbPotionsFoudreRealisables(array $ingredients): int
11    {
12        return 0;
13    }
14 }
```

Dans le dossier tests/Unit/, on crée le fichier de test PotionsTest.php reprenant l'ensemble de nos 9 scénarios de test avec les assertions PHPUnit :



```
1 <?php
2
3 namespace Tests\Unit;
4
5 use PHPUnit\Framework\TestCase;
6 use App\Potions;
7
8 class PotionsTest extends TestCase
9 {
10    public function test_2_ingredients_dans_les_quantites_requises(): void
11    {
12        $ingredients = ['nuages_tenebreux' => 2, 'poussiere_fee' => 1];
13        $this->assertEquals(1, Potions::nbPotionsFoudreRealisables($ingredients));
14    }
15
16    public function test_2_ingredients_mais_pas_assez_du_premier(): void
17    {
18        $ingredients = ['nuages_tenebreux' => 1, 'poussiere_fee' => 1];
19    }
20 }
```

On lance ensuite l'exécution de nos tests avec PHPUnit dans le terminal :

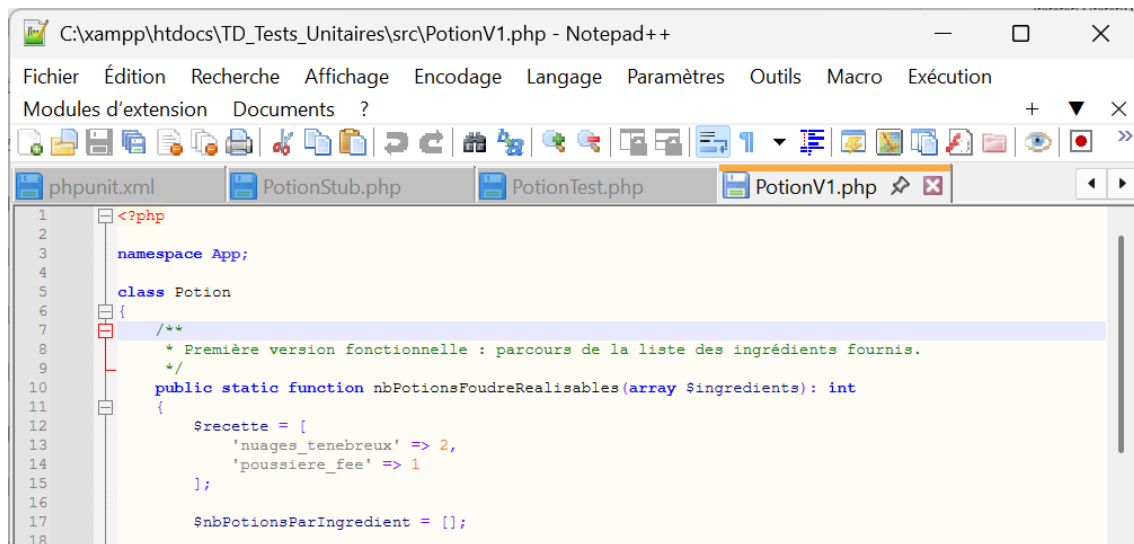


```
Console_TDD_Phase_Rouge 9 / 9 (100%)
F.F....FF
Time: 00:00.086, Memory: 8.00 MB
There were 4 failures:
1) Tests\Unit\PotionsTest::test_2_ingredients_dans_les_quantites_requises
Failed asserting that 0 matches expected 1.
C:\xampp\htdocs\TD_Testes_Unitaires\tests\Unit\PotionsTest.php:13
2) Tests\Unit\PotionsTest::test_2_ingredients_dans_les_quantites_x_fois_requises
Failed asserting that 0 matches expected 4.
```

Comme prévu, les tests sont 'rouges' : sur les 9 tests, 5 passent par hasard (ceux attendant 0 potion) et 4 échouent (ceux attendant 1 ou 4 potions). La phase rouge du TDD est validée.

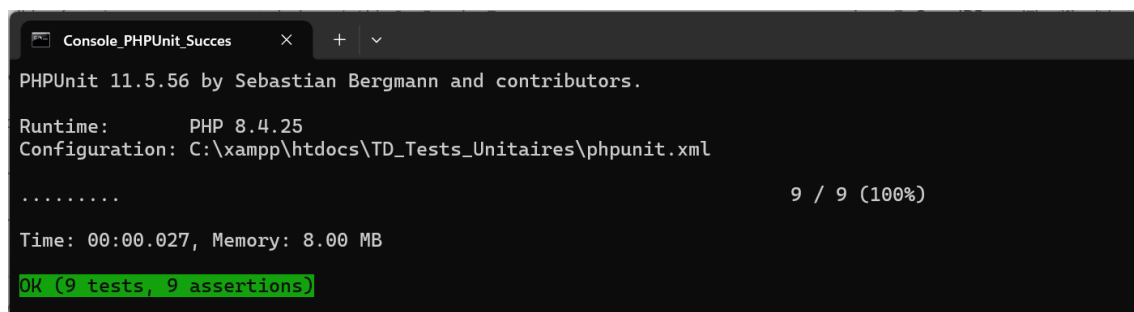
3) Phase verte du TDD : Première implémentation fonctionnelle

La deuxième phase du cycle TDD (Green) consiste à écrire le code nécessaire et suffisant pour faire passer tous les tests au vert. On implémente la logique en parcourant les ingrédients fournis et en vérifiant leur présence dans la recette :



```
1 <?php
2
3 namespace App;
4
5 class Potions
6 {
7     /**
8      * Première version fonctionnelle : parcours de la liste des ingrédients fournis.
9      */
10    public static function nbPotionsFoudreRealisables(array $ingredients): int
11    {
12        $recette = [
13            'nuages_tenebreux' => 2,
14            'poussiere_fee' => 1
15        ];
16
17        $nbPotionsParIngredient = [];
18    }
```

On relance immédiatement PHPUnit pour vérifier le résultat :



```
Console_PHPUnit_Succes
PHPUnit 11.5.56 by Sebastian Bergmann and contributors.

Runtime:      PHP 8.4.25
Configuration: C:\xampp\htdocs\TD_Tests_Unitaires\phpunit.xml

.....                                     9 / 9 (100%)

Time: 00:00.027, Memory: 8.00 MB

OK (9 tests, 9 assertions)
```

Tous les tests passent désormais avec succès (OK (9 tests, 9 assertions)). La fonction est officiellement opérationnelle.

4) Tests unitaires avec le framework PHPUnit

L'utilisation de PHPUnit nous permet de structurer nos tests sous forme de classes orientées objet héritant de PHPUnit\Framework\TestCase. Chaque cas de test est matérialisé par une méthode publique préfixée par test_ et utilise des assertions explicites telles que \$this->assertEquals(\$attendu, \$obtenu). Cela garantit une exécution rapide, déterministe et isolée de chaque cas de figure.

5) Tests unitaires avec le framework Pest

Nous reproduisons maintenant ces mêmes tests avec le framework Pest dans le fichier tests/Unit/PotionPestTest.php. Pest élimine le besoin d'écrire des classes et des méthodes verbeuses au profit de fonctions anonymes expressives test('description', function() { ... }) associées à l'API d'assertions fluide expect(...) :

```
1 <?php
2
3 use App\Potion;
4
5 test('2 ingrédients, dans les quantités requises', function () {
6     $ingredients = ['nuages_tenebreux' => 2, 'poussiere_fee' => 1];
7     expect(Potion::nbPotionsFoudreRealisables($ingredients))->toBe(1);
8 });
9
10 test('2 ingrédients, mais pas assez du premier', function () {
11     $ingredients = ['nuages_tenebreux' => 1, 'poussiere_fee' => 1];
12     expect(Potion::nbPotionsFoudreRealisables($ingredients))->toBe(0);
13 });
14
15 test('2 ingrédients, dans les quantités X fois requises', function () {
16     $ingredients = ['nuages_tenebreux' => 8, 'poussiere_fee' => 4];
17     expect(Potion::nbPotionsFoudreRealisables($ingredients))->toBe(4);
18 });
```

On exécute ensuite nos tests avec la commande `./vendor/bin/pest tests/Unit/PotionsPestTest.php` dans la console :

```
Console_Pest_Succes
PASS Tests\Unit\PotionsPestTest
✓ 2 ingrédients, dans les quantités requises
✓ 2 ingrédients, mais pas assez du premier
✓ 2 ingrédients, dans les quantités X fois requises
✓ manque l'ingrédient 1
✓ manque l'ingrédient 2
✓ ingrédients autres
✓ aucun ingrédient
✓ beaucoup ingrédient 1
✓ beaucoup ingrédient 2

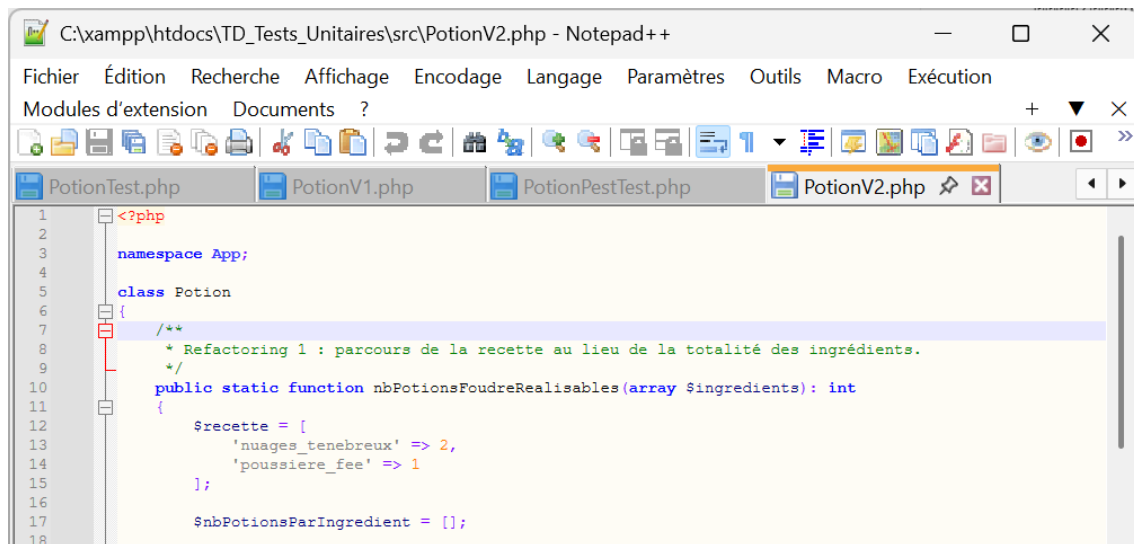
Tests: 9 passed (9 assertions)
Duration: 0.24s
```

L'affichage de Pest est particulièrement agréable et met en valeur chaque scénario testé avec une coche verte ✓ très lisible.

4) Refactoring du code et non-régression

1) Première optimisation : parcours ciblé de la recette

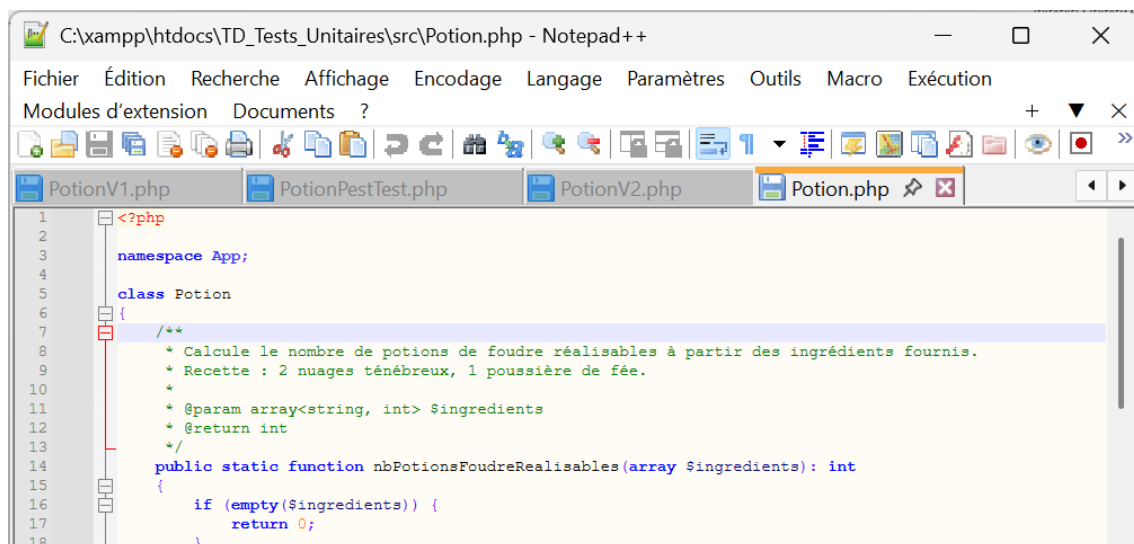
La troisième phase du cycle TDD (Refactor) consiste à améliorer la qualité et les performances du code sans modifier son comportement externe. Dans la première version, nous parcourions tous les ingrédients en stock. Si Harry possède 1000 ingrédients inutiles, la boucle effectue 1000 tours superflus. On optimise donc en parcourant uniquement les ingrédients nécessaires de la recette :



```
1 <?php
2
3 namespace App;
4
5 class Potion
6 {
7     /**
8      * Refactoring 1 : parcours de la recette au lieu de la totalité des ingrédients.
9      */
10    public static function nbPotionsFoudreRealisables(array $ingrédients): int
11    {
12        $recette = [
13            'nuages_tenebreux' => 2,
14            'poussiere_fee' => 1
15        ];
16
17        $nbPotionsParIngrédient = [];
18    }
```

2) Deuxième optimisation : calcul direct et fonction min()

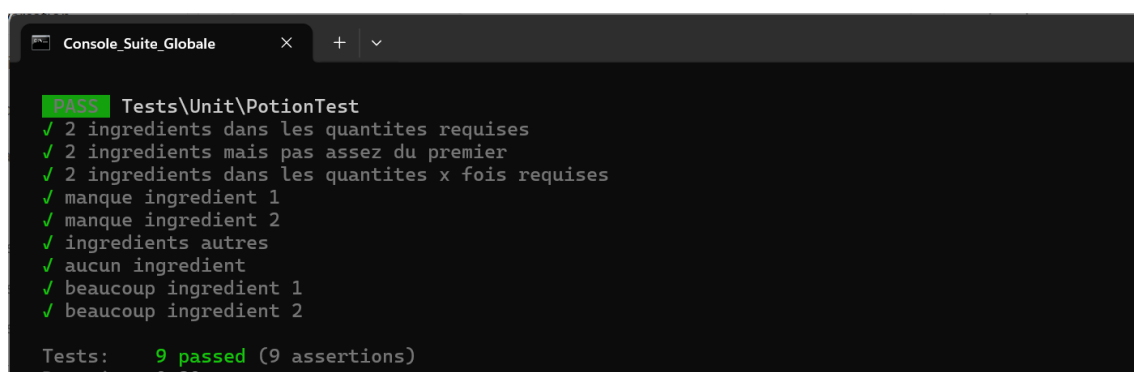
On pousse encore le refactoring en vérifiant immédiatement si le panier d'ingrédients est vide avec `empty($ingrédients)` pour retourner 0 au plus tôt. On calcule ensuite le minimum au fur et à mesure de la boucle grâce à la fonction `min()` :



```
1 <?php
2
3 namespace App;
4
5 class Potion
6 {
7     /**
8      * Calcule le nombre de potions de foudre réalisables à partir des ingrédients fournis.
9      * Recette : 2 nuages ténébreux, 1 poussière de fée.
10     */
11     @param array<string, int> $ingrédients
12     @return int
13     */
14    public static function nbPotionsFoudreRealisables(array $ingrédients): int
15    {
16        if (empty($ingrédients)) {
17            return 0;
18        }
19    }
```

3) Exécution globale et validation de la suite de tests

Grâce à notre filet de sécurité constitué par nos tests unitaires, nous pouvons modifier le code en toute sérénité. On lance la suite complète de tests via la commande `./vendor/bin/pest` pour vérifier la non-régression :



```
Console_Suite_Globale
[PASS] Tests\Unit\PotionTest
✓ 2 ingrédients dans les quantites requises
✓ 2 ingrédients mais pas assez du premier
✓ 2 ingrédients dans les quantites x fois requises
✓ manque ingredient 1
✓ manque ingredient 2
✓ ingrédients autres
✓ aucun ingredient
✓ beaucoup ingredient 1
✓ beaucoup ingredient 2

Tests: 9 passed (9 assertions)
Duration: 0.20s
```

L'ensemble des tests est validé avec succès. Aucune régression n'a été introduite durant les étapes de refactoring.

5) Conclusion

En conclusion, ce TD nous a permis de mettre en pratique concrètement les principes du TDD et des tests unitaires en PHP. Nous avons appris à installer et configurer un environnement de test professionnel avec Composer, PHPUnit et Pest. La comparaison entre PHPUnit et Pest met en évidence les atouts de chacun : PHPUnit constitue la référence éprouvée et rigoureuse en orienté objet, tandis que Pest apporte une syntaxe moderne, concise et ergonomique qui accélère la rédaction des tests. Enfin, nous avons expérimenté la puissance du refactoring sécurisé : disposer d'une suite de tests automatisée offre un confort de travail inégalé et garantit la robustesse et la pérennité du code en entreprise.